1 Introduction

In this assignment, it is expected that you explore autoencoders in reconstruction and denoising tasks, and exploit an object detector network with real-world data. This assignment is split into two parts: image reconstruction and object detection. On the object detector part you will also do a subtask about identifying and counting unique object instances across a video sequence (e.g., the number of unique people entering a classroom).

This assignment should be done in groups of 2 students, and it is part of your continuous evaluation. You should provide a report, with no more than 10 pages, and a notebook (.ipynb) which includes your development and analysis for the requested tasks. Structure your report in accordance with the assignment parts.

2 Dataset

In this assignment, you will be using the Kuzushiji-MNIST[1] dataset (similar to the MNIST dataset, used in the first CNN code) and a dataset acquired at the Department of Electrical and Computer Engineering (DEEC) and Department of Informatics Engineering (DEI) - University of Coimbra, for human detection.

The Kuzushiji-MNIST dataset is composed of 28×28 grayscale handwritten character images derived from classical Japanese literature. Designed as a drop-in replacement for the original MNIST dataset but with greater complexity, it contains 10 classes of cursive kana characters that are visually similar to one another. The full dataset includes 70,000 samples in total, with 60,000 images for training and 10,000 for testing, evenly distributed across the 10 character categories. The human detection dataset was recorded inside the DEEC and DEI buildings (MID-3K dataset), with a differential drive robot equipped with an RGB camera, a thermal camera and a LiDAR sensor. For the purpose of this work, we will be using the RGB set, containing 3,083 PNGs, 640×512 pixels (the images are not object-centric images), and it is available to download at https://github.com/kennedyk1/MID-3K-rgb. Note that an RGB image may contain more than one object/entity, or it may also appear without any recognizable elements (background image). The dataset contains at least 10,824 person bounding box instances in its 3,083 frames. Human instances are typically centered in the frame, as shown in the bounding-box center heatmap, and exhibit diverse sizes (see Fig. 1).

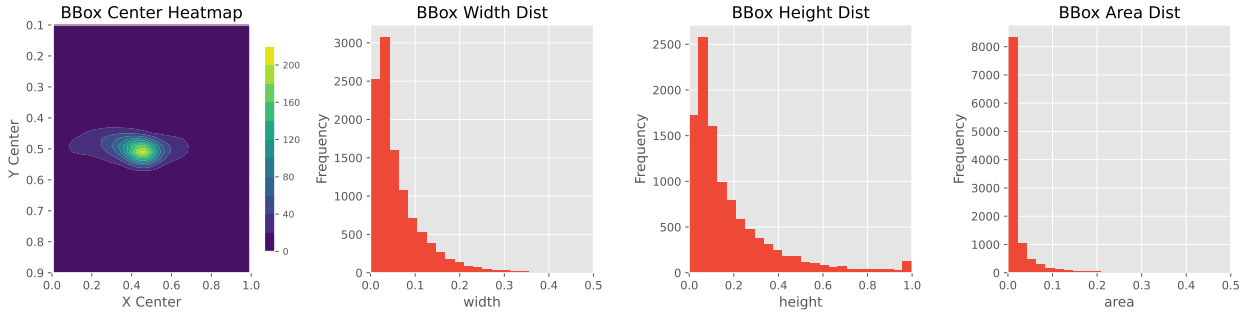


Figure 1: Combined dataset statistics including the spatial distribution of object centers (heatmap), along with the normalized width, height, and area distributions of all human instances.

3 Part I: Reconstruction and denoising using Autoencoders

3.1 Background Materials

In this first part of the assignment, you will define a suitable network composed by encoder and decoder architectures to process Kuzushiji-MNIST images. You will implement an Autoencoder (AE), a Variational

[1] https://github.com/rois-codh/kmnist

Autoencoder (VAE) and a Denoising Autoencoder (DAE).

3.1.1 Autoencoders

Implementing AEs is a simple process in PyTorch. Similarly to how a network is built, you must first define the encoder layer and decoder layer and later call them in the "**forward(self,input)**" function. You must also define a loss function that measures how similar (or how distant) the decoding and the input are. Commonly, Mean Squared Error (MSE) and Binary Cross Entropy (BCE) are used as loss functions for reconstruction problems. The training code from the previous assignment can be slightly modified to suit a reconstruction task, taking into account that the loss now considers the batch images and the batch decodings. It is important to note that the last activation function in the decoder layer plays an important role in the reconstruction task, as it should map the pixel values correctly. Although it is possible to create encoder/decoder layers using linear/fully connected layers, since we are working with image data, convolutional encoder/decoder layers should be used. You are not required to make the encoder and decoder a complete mirror of each other (e.g., MaxPool2D is not fully invertible), however, for most cases, "inverse" layers are required (Conv2d → ConvTranspose2d).

3.1.2 Variational Autoencoders

VAE is a type of generative model that combines elements of AEs and probabilistic modeling, meaning that from an initial AE network, modifications can be introduced to build a VAE network. The encoder layer of the AE, outputs a deterministic representation of the feature map, also known as the bottleneck. To implement a VAE, the first step will be to modify the encoder's output layer to output the mean (μ) and the log-variance ($\log \sigma^2$) of a probability distribution over the latent space. The problem now becomes how to sample from the distribution while maintaining the network traversable during backpropagation, as the efficient calculation of gradients during the backpropagation process is essential for training neural networks. This is known as the reparameterization trick and consists of adding a sampling layer that generates random samples taken from a normal distribution and scales it by the standard deviation (represented by the log-variance). The latent variable z is modeled as a Gaussian distribution with mean μ and standard deviation σ, and ϵ is a sample from a standard normal distribution ($\epsilon \sim \mathcal{N}(0, 1)$). Since the VAE learns the log-variance (as it improves numerical stability) the reparameterization trick is expressed as:

$$z = \mu + e^{0.5(\log \sigma^2)} \cdot \epsilon \tag{1}$$

The input of the VAE's decoder layer becomes the sampled latent vector. Finally, the total loss for the VAE is composed of two parts, the reconstruction loss and the KL Divergence loss. The reconstruction loss is the loss used in the AE. The KL Divergence loss is introduced to evaluate the distance between the learned distribution and a chosen prior (commonly a standard normal distribution). The KL Divergence loss is expressed as:

$$D_{KL} = -\frac{1}{2}\sum_{i=1}^{N}(1 - \mu_i^2 + \log \sigma_i^2 - e^{\log \sigma_i^2}) \tag{2}$$

where N is the dimensionality of the latent space.

3.1.3 Denoising Autoencoders

The main goal of a DAE is to learn how to denoise the input data, which can improve its ability to extract meaningful features. For that reason, the only step required is the generation of noisy inputs:

```
def add_noise(data, noise_factor=0.5):
    noisy_data = data + noise_factor * torch.randn_like(data)
    return torch.clamp(noisy_data, 0., 1.)
```

3.1.4 Evaluation Metrics

Since Autoencoders are data-driven, the evaluation of your models will be in terms of reconstruction errors and the qualitative representation of some reconstruction pairs.

3.2 Tasks

- Implement an AE network (the network of the first assignment can be a good starting point); You can use a fully-connected-based encoder-decoder, however a convolution-based encoder-decoder will be valued.
- Modify the encoder, add the reparameterization trick and add the VAE loss to the aforementioned network;
- Implement the DAE's pipeline;
- Implement the aforementioned evaluation metrics to evaluate your network;
- Gather the metrics and loss curves for all training conditions.
- Use t-SNE (t-distributed Stochastic Neighbor Embedding) to visualize the bottleneck/latent space of the AE, VAE and DAE. Example:

```
latent_space = []
labels = []
with torch.no_grad():
    for batch_idx, data in enumerate(training_loader):
        ae.eval()
        mu= ae.encode(data[0].to(device))
        latent_space.append(mu)
        labels.append(data[1])
latent_space = torch.cat(latent_space, dim=0).cpu().numpy()
labels = torch.cat(labels, dim=0).cpu().numpy()

from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, random_state=42)
latent_tsne = tsne.fit_transform(latent_space)

import matplotlib.pyplot as plt
plt.scatter(latent_tsne[:, 0], latent_tsne[:, 1], c=labels, cmap='viridis')
plt.colorbar()
plt.title('t-SNE Visualization of Latent Space')
plt.show()
```

- Apply transfer learning using the encoder layer of the AE, VAE and DAE. Add a classification layer to all the encoders (individual), freeze the encoder layer and only train the classification layer. Compare the classification results obtained, and how well the learned representations generalize.
- **EXTRA: Adapt/train the VAE network on the STL-10 dataset. Comment the results of the t-SNE and for a set of generated samples from the VAE latent space.**

4 Part II: Object Detection

4.1 Background Materials

In this second part of the assignment, you will use the YOLO object detector [2]. YOLO is one of the most successful real-time CNN-based object detectors developed to date.

4.1.1 Evaluation Metrics

Evaluating an object detection network is not the same as evaluating an object classification one. In an object detection network, the prediction results are bounding boxes within an image representing the detected objects, instead of a category label associated with an image. These predicted bounding boxes need to be evaluated by the Intersection over Union (IoU) metric, which evaluates the precision of each predicted bounding box with the ground-truth. IoU is represented by (3). The individual bounding box evaluation is the base of the object detection method evaluation metrics, designed by mean-Average Precision (mAP), which is based on Precision and Recall evaluation metrics as shown in (4), (5), (6), and (7).

[2] https://docs.ultralytics.com/

$$IoU = \frac{area\ of\ overlap}{area\ of\ union} \tag{3}$$

$$Precision = \frac{TP}{TP + FP} \tag{4}$$

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

$$AP = \frac{\sum_{r \in Recall([0,1])} Precision(r)}{|Recall([0,1])|} \tag{6}$$

$$mAP = \frac{\sum_{c \in C} APc}{|C|} \tag{7}$$

where:

- True Positive (TP): correct class prediction and usually IoU > 0.5;
- False Positive (FP): wrong class or IoU < 0.5;
- False Negative (FN): missed object detection.

4.1.2 Environment Setup

To download the dataset and the splits:
- First, download the set of images and labels:

```
git clone https://github.com/kennedyk1/MID-3K-rgb
```

- Second, download the train and test splits (you can later split the train into train+validation):

```
git clone https://github.com/luisgarrote/APA2026_MID-3K
```

Two splits are defined. The first with a random split composed of images stratified by department (DEEC+DEI) data and a second split of images captured on DEEC for training and images captured on DEI for testing.

For each image, a corresponding label file (.txt) is available, where each row will represent an object (bounding box) with the following structure (see Fig. 2):

```
object_class_id normalized_center_x normalized_center_y normalized_width normalized_height
```

The data is organized in the following structure, use the splits provided.

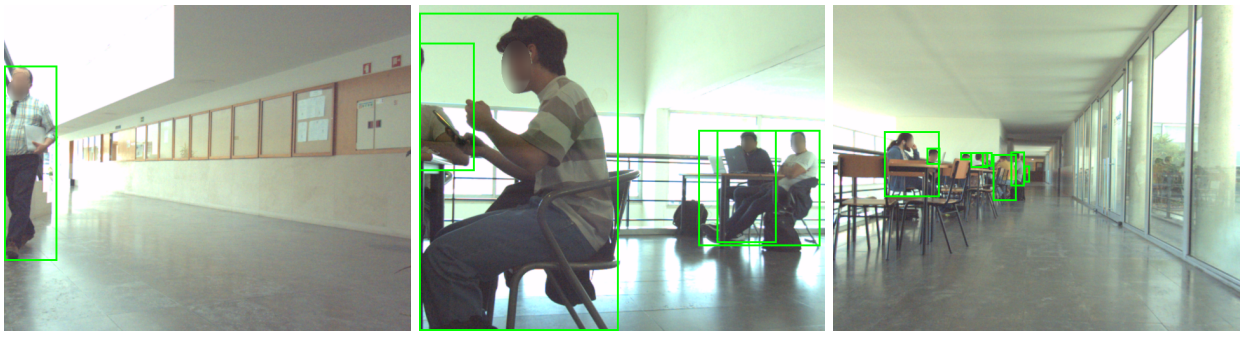


Figure 2: Samples of the MID-3K dataset. Ground-truth labels in green.

```
MID-3K-rgb/
    +-- images/
    |   +-- 000000_1715087159061715111.png
    |   +-- 000000_1715177279931855716.png
    |   +-- ...
    +-- labels/
    |   +-- 000000_1715087159061715111.txt
    |   +-- 000000_1715177279931855716.txt
    |   +-- ...
```

4.2 Tasks

The tasks are subdivided into Part II-A and Part II-B. Starting with Part II-A, the goal is to evaluate the performance of YOLO models in the provided random and DEEC_DEI splits.

- Prepare the data configuration files required to run/train the YOLO model on the provided set of images. One option is to set up a DATASETFILE (e.g., data/APA2026_DEEC_DEI.yaml) with (for the DEEC_DEI split, you need a separate YAML config file for the random split):

```
path: /PATH/MID-3K-rgb
train: /PATH/splits/DEEC_DEI/train.txt
val: /PATH/splits/DEEC_DEI/val.txt
test: /PATH/splits/DEEC_DEI/test.txt
nc: 1
names:
  0: person
```

- Select two different detection architectures available on the YOLO Ultralytics webpage (e.g., YOLOv8s, YOLOv10s or YOLO26s) and train the network. Train the networks from scratch during 25 epochs (it will take a while). To import and train a YOLO model, select the YOLO architecture (MODEL_NAME) and give it the DATASETFILE file to train the model. You can use the architecture YAML files defined by YOLO or use the pretrained runs use .pt checkpoints (scratch vs pretrain runs).

```
#!pip install ultralytics
from ultralytics import YOLO

model = YOLO('MODEL_NAME')
results = model.train(data=DATASETFILE,epochs=25, imgsz=640,  batch=batch)
```

- Train the same YOLO architectures, using as a starting point the pretrained checkpoints provided online[3] (also use 25 epochs).

- Gather the loss curves for all training conditions. Evaluate all the models in the corresponding testing set. Do not use the default loss curves on YOLO as they contain more elements that would require you to explain more concepts. To generate the curves and metrics for a specific model you can check:

```
model = YOLO('PATH_TO_MODEL.pt')
metrics = model.val(data=DATASETFILE,split='test')
```

For Part II-B, the goal is to combine your best detection model and extend the approach to identify and count unique object instances across a video sequence (e.g., the number of unique people entering a classroom). The video sequence is not provided, so you will need to capture it, taking into account that a reasonable number of persons entering a room or walking around is needed (> 10), with re-identification challenges (check Fig. 3). The video can be taken from a stationary position.

A solution that builds upon the work from Assignment 1 combined with the best detection model from Part II-A will receive a higher grade. However, you can explore solutions such as Object Counting in Ultralytics YOLO26[4]. A base solution will be using the detection model with object embeddings extracted from the detection crops. For highly dynamic scenarios you may need a tracking algorithm (e.g., SORT, Deep-SORT, BOT-SORT), meaning that you would do tracking and then check embeddings, and match them against other known embeddings (from already seen objects).

[3] https://docs.ultralytics.com/models/
[4] https://docs.ultralytics.com/guides/object-counting



Figure 3: Examples of how you can define the counting problem. In the first example every unique detection increases the counter (counter increments once for each newly identified person). In the second example, a line must be traversed for the counter to increase, it may be a simpler approach where analysis occurs only at the transition point.

5 Deliverables

Each group (two students) must deliver the following material:

- A detailed report (**cannot exceed 10 pages**) with:
  - **Part I**:
    - A theoretical comparison between each AE, VAE and DAE network. For that, present a diagram/table of each network architecture developed;
    - Report all hyperparameters used;
    - Plots of the attained loss curves, sets of images that represent the reconstruction capabilities of each network (input-reconstruction pairs) and t-SNE visualizations;
    - Classification architectures used and results obtained using each encoder's weights;
    - Comparison and detailed analysis of the results obtained using the developed AE, VAE and DAE architectures.
    - **Part II-A**:
      - A detailed step-by-step presentation training process;
      - A detailed analysis of the results obtained in the provided splits (Random and DEEC_DEI); Also, a comparison between achieved results by training the different YOLO models from scratch and by using transfer learning (from pretrained weights);
      - Are there differences in model generalization between departments (DEEC train vs DEI test) vs the random training? Explain why.
    - **Part II-B**:
      - A detailed step-by-step of the pipeline used for counting unique object instances across the captured video sequence;
      - You may need to train an additional model backbone for the re-identification stage (you can use a pretrained model for person re-ID). The process is similar to the one in the first assignment; in that case, provide the training protocol used.

  - Since this task requires that you count the unique embeddings for each object and keep them in memory for temporal associations (a possible solution is one embedding per new instance), provide a temporal embedding association map of the video (Person1, Person2, ..., Person5, Person1, Person2 - identify the re-identifications). You can use the package deface[5] to anonymize the data (GDPR) when including results in the report.

- Google Colab notebooks (.ipynb);

[5] https://github.com/ORB-HD/deface/tree/master